

CSE4356/CSE5392/EE4328/EE5315 System On Chip Design

Design of a I²C IP Core and Linux Device Drivers

Fall 2025

1 Overview

The goal of this project is to implement an I²C IP module on a Blackboard, capable of being controlled and configured from the hard processor subsystem over an AXI-4 lite interface. Implementation of Linux kernel modules is also required as part of the design.

2 I²C IP Hardware Interface

The I²C IP module will operate as the master on the bus and provide a simple hardware interface capable of supporting communication with multiple 7-bit address I²C devices at a 100 kbps rate. The interface will be connected to an external I²C peripheral for testing and verification. The I2C_SDA and I2C_SCL pins should be routed to a PMOD connector. The external I²C peripheral will be powered from the Blackboard.

A test pin (I2C_TEST_OUT) for outputting the 200 kbps clock enable should be routed to the PMOD connector as well.

3 I²C IP HPS Interface

The I²C IP module interfaces with the HPS using a memory-mapped interface. The following is the required memory map for the 5 word (20 byte) register space:

Register	Reg Offset	Bits	Field	Function
ADDRESS (R/W)	0	6:0		7-bit address of the device
REGISTER	4	7:0		8-bit register number
DATA (R/W)	8	7:0		Write: Write data to the TX FIFO Read: Reads data from the RX FIFO
STATUS (R/W1C)	12	0	RXFO	RX FIFO overflow (write 1 to clear)
		1	RXFF	RX FIFO full (auto clears when data is read)
		2	RXFE	RX FIFO empty (auto clears when data arrives)
		3	TXFO	TX FIFO overflow (write 1 to clear)
		4	TXFF	TX FIFO full (auto clears when data is sent)
		5	TXFE	TX FIFO empty (auto clears when data is written)
		6	ACK_ERROR	An ACK error occurred (write 1 to clear)
		7	BUSY	The module is busy transmitting or receiving
		31:8	DEBUG_IN	Debug interface (often used to peek at FSM)
CONTROL (R/W): reset 0	16	0	R/~W	Direction of data on bus (read=1, write=0)
		4:1	BYTE_COUNT	Number of bytes to be written or read
		5	USE_REGISTER	When reading and writing data from devices using internal registers, the register number should be sent after the address is sent.
		6	USE_REPEATED_START	1 = use repeated start conditional on read, 0 = use stop followed by start condition on reads

		7	START	Write: 1 = start bus transaction, 0 = no effect Read: 0
		8	TEST_OUT	1 = enable test output, 0 = disable test output
		23:9	-	<i>reserved</i>
		31:24	DEBUG_OUT	Debug interface (can be used to drive GPO or LED ports)

(W = write, R = read, W1C = write 1 to clear)

The register offset is relative to the AXI-4 lite interface aperture base address + 0x20000.

4 I²C IP Components

Clock divider:

An internal 200kHz clock enable signal provides the timing reference for I²C bit transitions. This allows the I2C_SDA and I2C_SCL signals to be changed on different edges. If TEST_OUT is set, then this signal is routed to a GPO pin.

Transmit FIFO:

The TX FIFO should be a 8-bit wide, 15-level deep FIFO (out of 16 active locations). The FIFO is written by writes to the DATA register. If the FIFO has no more room for data, the new data should be ignored. The TX FIFO outputs three status bits:

- The TXFF bit indicates that the FIFO is currently full. This bit is cleared by DATA register reads from the FIFO or by resetting the FIFO.
- The TXFO bit is set when a write occurs and TXFF is already set, indicating a loss of data. This bit is cleared upon resetting the FIFO or by writing '1' to the TXFO bit of the STATUS register.
- The TXFE bit indicates the FIFO is currently empty. This bit is set upon resetting the FIFO.

Receiver FIFO:

The RX FIFO should be a 8-bit wide, 15-level deep FIFO (out of 16 active locations). The FIFO is written after a data byte arrives on the I2C_SDA pin. When the software reads the DATA register, data is retrieved from the FIFO. If the FIFO has no more room for data, the new data should be ignored. The RX FIFO outputs three status bits:

- The RXFF bit indicates that the FIFO is currently full. This bit is cleared by reading from the FIFO or by resetting the FIFO.
- The RXFO is set when a write occurs and RXFF is already set, indicating a loss of data. This bit can be cleared upon resetting the FIFO or by writing '1' to the RXFO bit of the STATUS register.
- The RXFE bit indicates the FIFO is currently empty. This bit is set upon resetting the FIFO.

Transmitting Bytes:

Before sending data, the R/~W bit should be clear and the BYTE_COUNT, ADDRESS, and optionally the REGISTER (with USE_REGISTER set) should be valid. Then BYTE_COUNT number of bytes should be written to the DATA register. At some point (before, during, or after writes to the DATA register, transmission is started by setting the START bit and will continue until all bytes are transmitted. Check the ACK_ERROR bit to see if an error was detected.

Once transmission is started, the following events occur:

1. A START condition is sent on the bus.
2. The ADDRESS and '0' bit are sent on I2C_SDA with corresponding clocks on I2C_SCL.
3. An ~ACK bit is received on I2C_SDA.
4. If the ~ACK bit is not a zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.

5. If USE_REGISTER is set, the 8-bit REGISTER is sent on I2C_SDA with corresponding clocks on I2C_SCL and an ~ACK bit is received. If the ~ACK bit is not a zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.
6. BYTE_COUNT number of bytes are transmitted from the TX_FIFO on I2C_SDA with corresponding clocks on I2C_SCL. After each byte is sent, the ~ACK bit is received on I2C_SDA. If the ~ACK bit is not zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.
7. After all bytes are sent successfully, a STOP condition is sent on the bus.

Receiving Bytes:

Before receiving data, the R/~W bit should be set and BYTE_COUNT, ADDRESS, and optionally the REGISTER (with USE_REGISTER set) should be valid. Then the START bit is set to start the read and BYTE_COUNT number of bytes are read from the DATA register.

Once reception is started, the following events occur:

1. If USE_REGISTER is set, perform steps 2-7.
2. A START condition is sent on the bus.
3. The ADDRESS and '0' bit are sent on I2C_SDA with corresponding clocks on I2C_SCL.
4. An ~ACK bit is received on I2C_SDA.
5. If the ~ACK bit is not a zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.
6. The 8-bit REGISTER is sent on I2C_SDA with corresponding clocks on I2C_SCL and an ~ACK bit is received. If the ~ACK bit is not a zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.
7. If USE_REPEATED_START is clear, send a STOP condition.
8. Send a START condition.
9. The ADDRESS and '1' bit are sent on I2C_SDA with corresponding clocks on I2C_SCL.
10. An ~ACK bit is received on I2C_SDA.
11. If the ~ACK bit is not a zero, then set the ACK_ERROR condition, stop transmission, and send a STOP condition.
12. If BYTE_COUNT > 0, BYTE_COUNT-1 number of bytes are received on I2C_SDA and stored in the RX_FIFO. After each byte is received, an ACK (~ACK bit = 0) is sent on I2C_SDA.
13. If BYTE_COUNT > 0, a last byte is received on I2C_SDA and stored in the RX_FIFO. After the byte is received, a NACK (~ACK bit = 1) is sent on I2C_SDA.
14. After all bytes are received successfully, a STOP condition is sent on the bus.

Reset:

The FIFO, transmitter, and receiver should all accept a reset signal to place the hardware in a known state when the system is reset.

5 User-Space Memory Mapped Control

Using /dev/mem, write a simple application that runs in user space to configure the I²C IP module. It should be able to demonstrate that the I²C module operates correctly. You can use a protocol analyzer to help in this step and an external peripheral such as an MCP 23008 I²C bus expander or an MPU9250.

6 Kernel-Space Virtual File System (sysfs) Control

Write a kernel module (/sys/class/i2c) to control the I²C IP module. Adapt your C code or use a bash script to leverage the sysfs interface. You should support the following files and values:

File	Write value	Read value
/sys/kernel/i2c/mode	read or write	last written value
/sys/kernel/i2c/byte_count	0-15	last written value
/sys/kernel/i2c/register	0-255 or none	last written value

/sys/kernel/i2c/address	0-127	last written value
/sys/kernel/i2c/use_repeated_start	true or false	last written value
/sys/kernel/i2c/start	any value	current status
/sys/kernel/i2c/tx_data	value for TX_FIFO	N/A
/sys/kernel/i2c/rx_data	N/A	value in RX FIFO or -1 if empty

7 Bus Expander Control (CSE 5392, EE 5315, or Honors Contract)

Write a kernel module (/sys/class/i2c_expander) to control the I²C IP module, abstracting the functions of the I²C expander. Adapt C code or use a bash script to access this interface and display I²C expander information. You should support the following files and values at a minimum:

File	Write value	Read value
/sys/kernel/i2c_expander/ <i>pin</i> /dir	direction of port pin	last written value
/sys/kernel/i2c_expander/ <i>pin</i> /pullup	pull up control on port input	last written value
/sys/kernel/i2c_expander/ <i>pin</i> /data	data on port output	data on port pin

where */pin/* is the pin number (0-7 for MCP23008)

7 MPU9250 Control

This option will be discussed in class.

8 Testing

Your hardware will be tested in the ERB labs.

Computers and lab equipment will be provided on campus in ERB 125 for you to work on this project.

9 Deadlines

Project is due at the time indicated in the syllabus. An electronic copy of your complete project must be received by the graders prior to the start of defense time to receive points on the project. A written report (containing theory of operation and Verilog and C code) is required. Be sure to include a summary of your verification and debug environment and results (including waves from a simulator or a ILA). All projects are individual assignments.

10 Safety Issues

Please follow all UTA safety rules and those stated in the syllabus.

Please utilize the supervised lab resources in ERB 125 when working on the project for your safety. You may only use the resources in ERB 125 lab when the grader or other UTA staff is present.

Have fun!